

```
class Node:
    def __init__(self, data):
        self.data = data # Assigns the given data to the node
        self.next = None # Initialize the next attribute to null
```

```
class LinkedList:
    def __init__(self):
        self.head = None # Initialize head as None
    def insertAtBeginning(self, new_data):
        new_node = Node(new_data) # Create a new node
        new_node.next = self.head # Next for new node becomes the current head
        self.head = new_node # Head now points to the new node
    def insertAtEnd(self, new_data):
        new_node = Node(new_data) # Create a new node
        if self.head is None:
            self.head = new_node # If the list is empty, make the new node the head
            return
        last = self.head
        while last.next: # Otherwise, traverse the list to find the last node
            last = last.next
        last.next = new_node # Make the new node the next node of the last
    def insertAtSpecificPositionInMiddle(self, new_data, position):
        new_node = Node(new_data) # Create a new node
        current = self.head
        count = 0
        while current and count < position - 1:
            current = current.next
            count += 1

        # Check if the position is valid (i.e., we didn't run off the end prematurely)
        if current is None:
            print("Position out of bounds. Node not inserted.")
            return

        # Adjust the pointers for insertion
        new_node.next = current.next
        current.next = new_node
    def deleteFromBeginning(self):
        if self.head is None:
            return "The list is empty" # If the list is empty, return this string
        self.head = self.head.next # Otherwise, remove the head by making the next node the new head
    def deleteFromEnd(self):
        if self.head is None:
            return "The list is empty"
        if self.head.next is None:
            self.head = None # If there's only one node, remove the head by making it None
            return
        temp = self.head
        while temp.next.next: # Otherwise, go to the second-last node
            temp = temp.next
        temp.next = None # Remove the last node by setting the next pointer of the second-last node to None
    def deleteFromAnyPositionInMiddle(self, position):
        current = self.head
        count = 0
        # Traverse to the node *before* the target position
        while current is not None and count < position - 1:
            current = current.next
            count += 1

        # If the position is out of bounds or the current node is the last node
        if current is None or current.next is None:
            print("Position out of range.")
            return

        # Update pointers to skip the node to be deleted
        node_to_delete = current.next
        current.next = node_to_delete.next
        # The 'node_to_delete' is now unreferenced and garbage collected
    def printList(self):
        temp = self.head # Start from the head of the list
        while temp:
            print(temp.data, end=' ') # Print the data in the current node
            temp = temp.next # Move to the next node
        print() # Ensures the output is followed by a new line
```

```

l1 = LinkedList()

# Insert words at the beginning
l1.insertAtBeginning('fox')
l1.insertAtBeginning('brown')
l1.insertAtBeginning('quick')
l1.insertAtBeginning('the')

# Insert a word at the end
l1.insertAtEnd('jumps')

# Print the list before deletion
print("List before deletion:")
l1.printList()

# Deleting nodes from the beginning and end
l1.deleteFromBeginning()
l1.deleteFromEnd()

# Print the list after deletion
print("List after deletion:")
l1.printList()

List before deletion:
the quick brown fox jumps
List after deletion:
quick brown fox

```

```

l1.insertAtSpecificPositionInMiddle('cunning',2)
l1.printList()

quick brown cunning fox

```

```

l1.deleteFromAnyPositionInMiddle(2)
l1.printList()

quick brown fox

```

A **linked list** is a linear data structure composed of a sequence of nodes, where each node contains data and a pointer (or reference) to the next node in the sequence. Unlike arrays, linked list elements are **not stored in contiguous memory locations**, but are dynamically allocated as needed.

Core Concepts

Node: The fundamental building block of a linked list, typically consisting of two parts: **Data/Payload:** The actual value or information stored in the node. **Pointer/Link:** The memory address or reference to the next node in the sequence. The pointer of the last node in a linear list typically points to **NULL** to signify the end. **Head:** A pointer that points to the first node of the linked list. Access to the entire list is generally facilitated through the head pointer. **Tail:** Refers to the last node in the list. In a singly linked list, it points to NULL, but in some implementations (like a queue or circular list), a tail pointer is maintained for efficient operations.

Types of Linked Lists

Linked lists come in several variations to suit different application needs:

- **Singly Linked List:** The simplest type, where each node has a pointer onlyList item to the next node, allowing traversal in only one direction (forward).
- **Doubly Linked List:** Each node contains pointers to both the next and the previous nodes in the sequence, enabling traversal in both forward and backward directions.
- **Circular Linked List:** A variation where the last node's pointer points back to the first node, forming a continuous loop without a NULL end. This is useful for applications that require continuous cycling.

Common Operations Common operations performed on a linked list include:

Insertion: Adding a new node to the list (at the beginning, end, or a specific position).

Deletion: Removing a specific node from the list.

Traversal: The process of visiting each node in the list sequentially, starting from the head.

Searching: Locating a specific node or value within the list.

